Locking

Basic locking idea

Obtain exclusive lock on object before using it.

Release lock when done.

Write- and read-locks

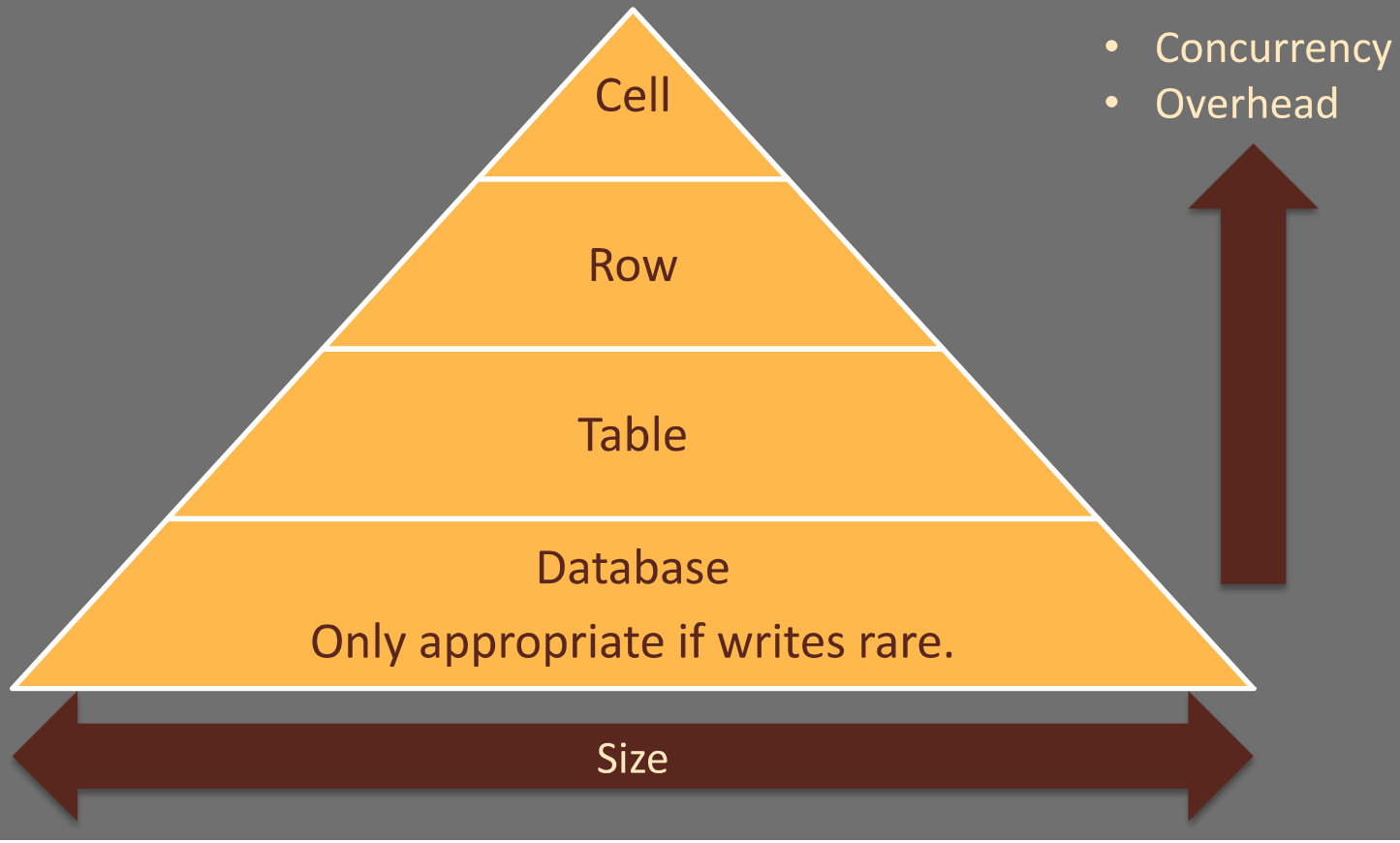
Obtain X (exclusive) lock on object before writing.

- No other transaction can get X or S lock.

Obtain S (shared) lock on object before reading.

- No other transaction can get X lock.

Lock what size of objects?



- Concurrency
- Overhead

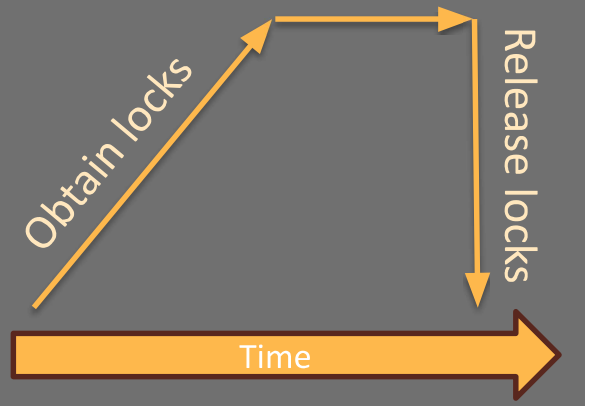
Different protocols possible

Obtain and release locks

- as early or late as possible?
- in a specific order?

Rigorous two-phase locking

- Obtain X/S lock before write/read.
- Release locks when transaction completes.



- Standard practice in RDBMS.
- Guarantees conflict serializability
  - Except: deadlock
- Conceptually straight-forward to implement
- Transparent to user

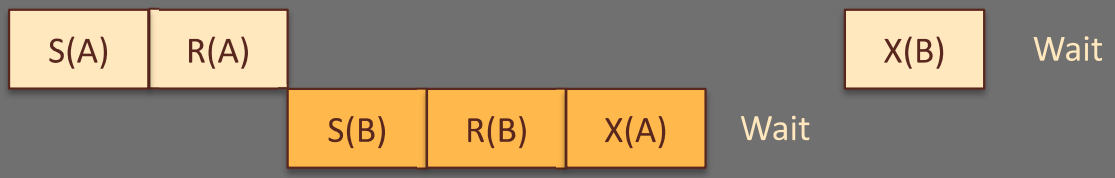
Serializability

Theorem: R2PL only allows schedules with acyclic conflict graph.

Proof outline: If $T_i \rightarrow T_j$, then T_j must wait for T_i to complete. Cannot have $T_j \rightarrow T_i$.

Corollary: R2PL only allows serializable schedules.

Allows deadlock

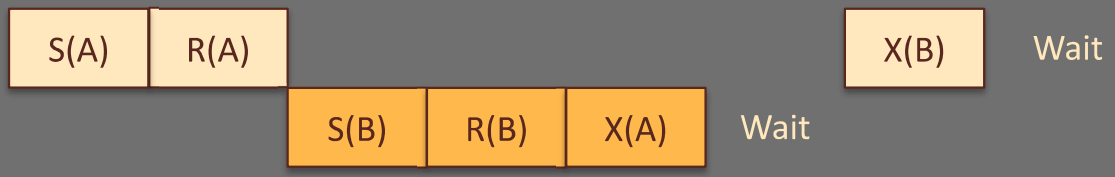


Deadlock = Cycle of transactions waiting for each other.

Dealing with deadlock

Prevent or detect & fix?

Deadlock detection



Periodically check Wait-for graph for cycles. Abort one of the deadlocked transactions.

Etc.

Other types of locks possible

- Range
- Update
- Table schema
- …

PostgreSQL allows user-controlled locks.

Locks & isolation levels

Isolation level	Write locks	Read locks	Range locks
Read uncommitted	No	No	No
Read committed	Released at end of transaction	Released at end of current operation	No
Repeatable read	Released at end of transaction	Released at end of transaction	No
Serializable	Released at end of transaction	Released at end of transaction	Released at end of transaction